



CHRIST

(DEEMED TO BE UNIVERSITY)

BANGALORE | DELHI NCR | PUNE

ORB: A GOVERNANCE CONSOLE FOR LOCAL AND CLOUD LANGUAGE MODELS

By

Praneeth M (2547142)

Parthiv Sushil (2547141)

Hari Prasad (2547120)

Under the guidance of

Dr. Somnath Sinha

Specialization Project report submitted in partial fulfilment of the requirements of IV

Trimester MCA,

CHRIST (Deemed to be University)

August – 2026



CHRIST

(DEEMED TO BE UNIVERSITY)

BANGALORE | DELHI NCR | PUNE

CERTIFICATE

*This is to certify that the report titled **Orb: A Governance Console for Local and Cloud Language Models** is a bona fide record of work done by Praneeth M (2547142), Parthiv Sushil (2547141) and Hari Prasad (2547120) of CHRIST (Deemed to be University), Bangalore, in partial fulfilment of the requirements of IV Trimester MCA during the year 2026*

Head of the Department

Project Guide

Valued by:

1. Name : Praneeth M

2. Register Number : 2547142

Examination Centre : CHRIST (Deemed to be University)

Date of Exam :

AI USAGE DECLARATION STATEMENT

We hereby declare that Artificial Intelligence tools were used only for supportive purposes such as idea exploration, conceptual understanding, literature guidance, and grammar refinement during the development of this project. No AI generated code has been copied or submitted, and all programming, system development, experimentation, analysis, and results were independently developed and implemented by our team. We confirm that this project is our original work and reflects our own understanding and effort, and we take full responsibility for its authenticity and adherence to academic integrity policies.

We also declare that the approximate usage of Artificial Intelligence tools for supportive purposes in this project is _____%, limited to idea exploration, literature search guidance, grammar refinement, clarification of technical concepts, and other academic assistance.

The details of AI tools used in this project and their purpose are listed below:

1. **Name of the AI Tool:** Claude (Anthropic)
Purpose of Use: Ideation and conceptual assistance
2. **Name of the AI Tool:** ChatGPT (OpenAI)
Purpose of Use: Clarification of technical concepts
3. **Name of the AI Tool:** Grammarly
Purpose of Use: Grammar and spell check

This declaration confirms that the AI tools mentioned above were used only for supportive and assistive purposes and that no AI generated code has been directly copied or submitted as part of this project.

Name of the Students : Praneeth M, Parthiv Sushil, Hari Prasad

Register Numbers : 2547142, 2547141, 2547120

Project Title : Orb: A Governance Console for Local and
Cloud Language Models

Signature :

Date :

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Dr. Somnath Sinha**, Department of Computer Science, CHRIST (Deemed to be University), for his constant guidance, encouragement and valuable suggestions throughout the course of this project. His insights into system design and research discipline shaped this work at every stage.

We are grateful to the **Head of the Department** and all faculty members of the Department of Computer Science for providing the infrastructure and academic environment necessary to carry out this project, and to the panel members of the CIA reviews whose feedback helped us refine both the system and this report.

We also thank our classmates and families for their patience and support, and we acknowledge the open-source communities behind Node.js, Express, Next.js, Ollama and SQLite, whose software forms the foundation on which Orb is built.

ABSTRACT

Large language models are increasingly executed on personal machines, where they can invoke tools that read files, run shell commands and access the web, yet users have little visibility into or control over these actions. Orb addresses this gap as a governance-first command console for local and cloud language models. The system places a governed Express/TypeScript gateway between the user and the model: it routes chat requests either to local Ollama models or to cloud providers (Anthropic, OpenAI, Groq, Mistral), enforces a per-tool execution policy of automatic, policy-based or manual approval, streams responses to a Next.js frontend, and persists sessions, audit logs, provider connector keys and extracted memory facts in SQLite. Every programmatic request through the Bearer-token API is fully audited with request messages, tool calls, policy decisions, latency and token counts. An asynchronous post-chat analysis stage extracts durable user facts into a per-user memory store and computes a hallucination-risk score for each response. Authentication is handled by Clerk for interactive users and by hashed API keys for external clients. The result is a working oversight layer that demonstrates how policy enforcement, auditability and telemetry can be added around both local and cloud model inference without modifying the models themselves.

This report documents the background, system analysis, requirement specification and design of Orb in its current state. It covers the existing-system landscape and its limitations, the proposed system and its features, the functional and non-functional requirements, the system architecture, data flow and database design, the interface design of the console, and the implementation, testing and results of the working system, followed by the conclusion, future enhancements, user manual and project planning.

TABLE OF CONTENTS

AI Usage Declaration Statement	iii
Acknowledgement	iv
Abstract	v
List of Figures	viii
List of Tables	ix
1 Introduction	1
1.1 Overview	1
1.2 Problem Statement	1
1.3 Objectives of the Project	2
1.4 Scope of the Project	3
1.5 Organisation of the Report	3
2 System Analysis	4
2.1 Existing System	4
2.1.1 Limitations of the Existing System	4
2.2 Proposed System	4
2.2.1 Advantages of the Proposed System	5
2.3 Feasibility Study	5
2.3.1 Technical Feasibility	5
2.3.2 Economic Feasibility	6
2.3.3 Operational Feasibility	6
3 System Requirements Specification	7
3.1 Functional Requirements	7
3.2 Non-Functional Requirements	8
3.3 Hardware Requirements	9
3.4 Software Requirements	9
3.5 Technologies Used	9
3.5.1 Next.js and React	9
3.5.2 Express and TypeScript	10
3.5.3 SQLite with better-sqlite3	10
3.5.4 Clerk	10
3.5.5 Ollama and Cloud Providers	10
4 System Design	11
4.1 Architecture Overview	11
4.2 Data Flow Diagrams	11
4.2.1 Context Level (Level 0)	11
4.2.2 Level 1 Decomposition	11
4.3 Database Design	12
4.4 Process Design: The Agent Loop	13

4.5	Connector Key Resolution	14
4.6	API Design	14
4.7	User Interface Design	14
5	Implementation	16
5.1	Implementation Environment	16
5.2	Module Description	16
5.2.1	Authentication Module	16
5.2.2	LLM Abstraction and Factory	16
5.2.3	Agent Loop and Policy Engine	17
5.2.4	Tool Executor	17
5.2.5	Session Management Module	17
5.2.6	Memory and Risk Analysis Module	17
5.2.7	Connectors Module	18
5.2.8	API Keys, Audit and Analytics Module	18
5.2.9	Frontend Console	18
6	Testing	19
6.1	Testing Strategy	19
6.2	Test Cases	19
6.3	Testing Outcome	19
7	Results and Discussion	21
7.1	Results	21
7.2	Discussion	21
8	Conclusion and Future Enhancements	24
8.1	Conclusion	24
8.2	Future Enhancements	24
	References	26
A	User Manual	27
A.1	Installation	27
A.2	Using the Console	27
A.3	Programmatic Access	27
B	Planning	29
B.1	Gantt Chart	29
C	Design Concepts Analysis	30
C.1	Health and Safety	30
C.2	Privacy and Data Protection	30
C.3	Security	30
C.4	Sustainability and Cost	30

LIST OF FIGURES

4.1	Context-level data flow diagram	12
4.2	Entity–relationship diagram	12
4.3	Connector key resolution sequence	14
7.1	Landing screen	21
7.2	Chat console with telemetry and policy controls	22
7.3	Compact chat view	23

LIST OF TABLES

3.1	Hardware requirements	9
3.2	Software requirements	9
4.1	Database tables and their purpose	13
4.2	Principal API endpoints	15
6.1	Representative test cases	20
B.1	Project schedule	29

CHAPTER 1: INTRODUCTION

1.1 OVERVIEW

Language models have moved from remote, provider-hosted services to software that runs directly on personal computers. Runtimes such as Ollama make it possible to serve models like Qwen 3 and Gemma 3 on a laptop, while cloud providers such as Anthropic, OpenAI, Groq and Mistral expose increasingly capable models through public APIs. In both cases, modern usage goes beyond plain text generation: models are given *tools* — the ability to read and write files, execute shell commands and search the web — and are run in an agentic loop where the model decides which tool to call next.

This shift creates a governance problem. A model that can execute bash commands or read arbitrary files on the host machine is effectively an autonomous actor on that machine, yet common chat interfaces provide no policy layer, no audit trail and no measure of how trustworthy a given answer is. **Orb** is our response to this problem: a web-based governance console that sits between the user and the model. All inference — local or cloud — passes through a governed gateway that enforces execution policy on every tool call, records detailed audit logs, tracks latency and token telemetry, scores responses for hallucination risk, and maintains long-term, per-user memory, all behind authenticated access.

1.2 PROBLEM STATEMENT

Existing interfaces to local and cloud language models suffer from the following gaps:

- **No execution governance.** When a model requests a tool action (file read, file write, shell command, web search), interfaces either execute it silently or refuse it entirely; there is no per-tool policy of allow, block or require-human-approval.

- **No auditability.** There is no durable record of what was asked, which tools ran, what they returned and what policy decisions were taken — a prerequisite for any responsible deployment.
- **No trust signal.** Users receive fluent answers with no indication of how likely the response is to be fabricated.
- **Fragmented provider access.** Local models and each cloud provider come with separate interfaces, credentials and conventions; there is no single governed entry point.
- **No durable context.** Facts a user states in one conversation are forgotten in the next unless the user repeats them.

The problem, therefore, is to design and implement a single oversight layer that provides policy enforcement, auditing, telemetry, risk scoring and memory across heterogeneous model backends.

1.3 OBJECTIVES OF THE PROJECT

- To build a unified, authenticated web console from which a user can converse with local Ollama models and cloud models (Anthropic, OpenAI, Groq, Mistral) through one governed gateway.
- To enforce a per-tool execution policy — Auto, Policy or Manual approval — on every tool call made by a model, with the ability to pause an agent run until a human approves.
- To provide a programmatic, Bearer-token API (/api/v1/*) whose every call is written to an audit log with request messages, tool calls, tool results, policy decisions, latency and token counts.
- To compute a hallucination-risk score for each response and expose per-session telemetry (latency, first-token time, blocked actions).

- To extract durable user facts from conversations into a per-user memory store that is automatically injected into future system prompts.
- To manage provider credentials centrally through a connectors module in which database-stored keys override environment variables.
- To persist sessions so that conversations can be listed, inspected and resumed with their full message history, policies and settings.

1.4 SCOPE OF THE PROJECT

Orb targets a single-machine, local-first deployment: the gateway, database and frontend run on the user's computer, and local inference never leaves the machine. The system supports five provider connectors (Anthropic, OpenAI, Groq, Mistral for chat; Google for key storage), four governed tools (file-system read, file write, shell execution, web search via Tavily), interactive use through a browser and programmatic use through API keys. Multi-node deployment, fine-tuning of models, and enforcement *inside* the model runtime are outside the present scope and are discussed as future enhancements in Chapter 8.

1.5 ORGANISATION OF THE REPORT

Chapter 2 analyses the existing systems and establishes feasibility. Chapter 3 specifies functional, non-functional, hardware and software requirements. Chapter 4 presents the system design — architecture, data-flow diagrams, database schema and sequence flows. Chapter 5 describes the implementation of each module. Chapter 6 covers the testing strategy and test cases. Chapter 7 presents the results with screenshots of the working system. Chapter 8 concludes the report and lists future enhancements, followed by references and appendices.

CHAPTER 2: SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

Interaction with language models today occurs mainly through three kinds of software. Provider web chat applications (e.g., provider consoles for GPT or Claude) are polished but cloud-only: every prompt leaves the user's machine, no local models can be used, and the user has no control over tool execution policy. Local runtimes such as Ollama expose a REST endpoint on `localhost:11434` and simple CLI chat, but offer no authentication, no persistence beyond the terminal, no policy layer and no telemetry. General-purpose LLM front-ends add chat history on top of local runtimes but treat the model as a text generator only — when tool use is supported at all, it is executed without per-tool governance, and none of them produce an audit trail suitable for review.

2.1.1 Limitations of the Existing System

- Tool calls (shell, file access, web) execute without a policy gate or human-approval checkpoint.
- No unified interface spans both local models and multiple cloud providers with centrally managed credentials.
- No audit log records requests, tool activity and decisions.
- No indicator of response reliability is shown to the user.
- Conversations are either lost or stored without any associated telemetry, and facts about the user are not carried forward.

2.2 PROPOSED SYSTEM

The proposed system, Orb, is a governance console composed of a Next.js frontend and an Express/TypeScript gateway backed by SQLite. All model traffic flows through the gateway,

which adds five capabilities missing from existing systems: (i) a policy engine that resolves every tool call to *Allowed*, *Blocked* or *Requires Approval*, pausing the agent loop for manual approval; (ii) full audit logging of programmatic API traffic; (iii) telemetry — per-message latency, first-token time and a hallucination-risk index computed asynchronously after each reply; (iv) per-user memory extraction so that durable facts are injected into future prompts; and (v) a connectors module that stores provider API keys in the database, overriding environment variables, so that local and cloud models are reached through one interface.

2.2.1 Advantages of the Proposed System

- Human-in-the-loop control over dangerous operations such as shell execution and file writes.
- Complete, queryable audit history for programmatic access.
- A trust signal (risk score) attached to responses.
- Single governed entry point for five providers plus local inference, with API keys managed centrally.
- Local-first privacy: local inference and all governance data remain on the user's machine.

2.3 FEASIBILITY STUDY

2.3.1 Technical Feasibility

The system is built entirely from mature, freely available technologies: Node.js with Express and TypeScript for the gateway, Next.js with React for the frontend, SQLite (via `better-sqlite3`) for storage, Clerk for identity, and Ollama for local inference. All components run on commodity hardware; local models such as Qwen 3 and Gemma 3 run acceptably on a machine with 8–16 GB of RAM, and the gateway itself is lightweight. No component requires specialised infrastructure, so the project is technically feasible.

2.3.2 Economic Feasibility

Every framework and library used is open source and free of licence cost. Local inference through Ollama incurs no per-token charges; cloud providers are optional and pay-as-you-go, with keys supplied by the user. Development required only the team's existing laptops. The project is therefore economically feasible for an academic setting.

2.3.3 Operational Feasibility

The console runs in a standard web browser with a familiar chat interface, so no special training is needed for basic use. Policy configuration and API-key management are presented through dedicated screens with sensible defaults (safe tools auto-allowed, dangerous tools requiring approval). Because the system is self-hosted and local-first, it fits environments where data cannot leave the machine, which widens rather than narrows its operational applicability.

CHAPTER 3: SYSTEM REQUIREMENTS SPECIFICATION

3.1 FUNCTIONAL REQUIREMENTS

- **FR-1 Authentication.** The system shall authenticate interactive users via Clerk session cookies and programmatic clients via `orb_sk_...` Bearer API keys verified by SHA-256 hash lookup.
- **FR-2 Chat.** The system shall stream chat responses as NDJSON from local Ollama models or cloud providers selected by the user, within an agentic loop that supports tool calls.
- **FR-3 Policy enforcement.** The system shall support three chat modes — *Auto* (all tools allowed), *Manual* (every tool call requires approval) and *Policy* (per-tool rules of Allowed / Blocked / Requires Approval) — and shall pause the loop, emitting a `requires_approval` event, when approval is needed; approved calls resume via a dedicated endpoint. Blocked calls shall be reported back to the model as refusals.
- **FR-4 Tool execution.** The system shall provide governed tools for file read, file write, directory listing, shell execution and Tavily web search.
- **FR-5 Sessions.** The system shall persist every conversation with its messages, policies and settings; users shall be able to list, inspect, complete and resume sessions.
- **FR-6 Memory.** The system shall asynchronously extract durable user facts after each exchange, store them per user, inject them into future system prompts, and allow the user to view and delete them.
- **FR-7 Risk scoring.** The system shall compute a hallucination-risk score for each assistant reply and attach it to the persisted message.
- **FR-8 API keys and audit.** The system shall create, list, revoke and re-scope API keys with per-key tool flags (`fs`, `bash`, `web`), and shall write an audit row for every

/api/v1/chat call containing request messages, response, tool calls/results, policy decisions, latency, token counts and status code.

- **FR-9 Analytics.** The system shall summarise audit activity (volume, blocked and error counts, latency, hourly buckets, top tools, anomalies) over a selectable window.
- **FR-10 Connectors.** The system shall store provider API keys for Anthropic, OpenAI, Groq, Mistral and Google, mask them on read, and resolve keys with database values overriding environment variables.
- **FR-11 Model discovery.** The system shall list locally installed Ollama models and detect host CPU/RAM to recommend a performance mode.

3.2 NON-FUNCTIONAL REQUIREMENTS

- **NFR-1 Privacy.** Local inference and all governance data (sessions, memories, audit logs, keys) shall remain on the user's machine in a local SQLite file.
- **NFR-2 Responsiveness.** Responses shall be streamed token-by-token; first-token time and total latency shall be measured and displayed.
- **NFR-3 Security.** API keys shall be stored only as SHA-256 hashes with last-four digits for display; provider keys shall be masked whenever read back; dangerous tools shall default to manual approval.
- **NFR-4 Reliability.** The database shall run in WAL mode; post-chat analysis shall be fire-and-forget so that a failure in analysis never breaks the chat path.
- **NFR-5 Usability.** All governance features shall be reachable from a single-page console with dedicated screens for chat, sessions, memory, API keys and connectors.
- **NFR-6 Maintainability.** The backend shall be written in typed TypeScript with modular routes, repositories and a provider-agnostic LLM interface so that new providers can be added by implementing one class.

3.3 HARDWARE REQUIREMENTS

Table 3.1 Hardware requirements

Component	Minimum specification
Processor	64-bit quad-core CPU (Apple Silicon / Intel i5 or better)
RAM	8 GB (16 GB recommended for larger local models)
Storage	20 GB free (local model weights are 2–8 GB each)
Display	Any modern display; browser-based UI
Network	Required only for cloud providers, Clerk and web search

3.4 SOFTWARE REQUIREMENTS

Table 3.2 Software requirements

Category	Software
Operating system	macOS / Linux / Windows
Runtime	Node.js 20+
Backend	Express 4, TypeScript, tsx; better-sqlite3 13
Frontend	Next.js 16 (App Router), React 19, Framer Motion, GSAP
Database	SQLite (WAL mode), file backend/data/orb.db
Authentication	Clerk (@clerk/nextjs, @clerk/express)
Local inference	Ollama (models: Qwen 3, Gemma 3)
Cloud SDKs	@anthropic-ai/sdk; OpenAI-compatible HTTP for OpenAI / Groq / Mistral
Web search	Tavily (@tavily/core)
Tools	Git, VS Code, npm

3.5 TECHNOLOGIES USED

3.5.1 Next.js and React

The frontend is a single-page Next.js App-Router application. React 19 renders the console’s screens (landing, chat, sessions, memory, API, connectors), while Framer Motion and GSAP

provide the animated transitions. The frontend communicates with the gateway over `fetch` with credentials included, consuming NDJSON streams for chat.

3.5.2 Express and TypeScript

The gateway is an Express 4 application written in TypeScript and run with `tsx`. Routes are grouped by resource (chat, sessions, memories, keys, connectors, models, system) with two authentication middlewares: `requireAuth` (Clerk cookie) for interactive routes and `apiKeyAuth` (Bearer hash lookup) for `/api/v1/*`.

3.5.3 SQLite with better-sqlite3

Storage is a single SQLite file opened in WAL mode through the synchronous `better-sqlite3` driver — an appropriate choice for a single-writer, local-first application. Structured values (messages, settings, policies, tool flags) are stored as JSON columns, a deliberate denormalisation for low-concurrency local use.

3.5.4 Clerk

Clerk is the external identity provider: sign-in and sign-up pages are Clerk components in the frontend, and the backend verifies session cookies via `@clerk/express`. There is no local users table; `user_id` columns hold Clerk user IDs.

3.5.5 Ollama and Cloud Providers

Local inference uses Ollama's streaming chat API on `localhost:11434`. Cloud inference is abstracted behind a common LLM interface with three implementations: Ollama, Anthropic (official SDK) and an OpenAI-compatible client reused for OpenAI, Groq and Mistral. A factory selects the implementation from the chosen model and resolves the provider key through the connectors module.

CHAPTER 4: SYSTEM DESIGN

4.1 ARCHITECTURE OVERVIEW

Orb follows a three-tier, gateway-centred architecture. The presentation tier is the Next.js console; the application tier is the Express gateway containing authentication, the agent loop, the policy engine, tool executor, connectors resolver and post-chat analysis; the data tier is SQLite. External actors are the browser user (Clerk-authenticated), third-party API clients (key-authenticated), Clerk, local Ollama, the four cloud chat providers and the Tavily search API. The defining property of the architecture is that *no model call and no tool call can bypass the gateway*: policy, audit and telemetry are applied at the single choke point through which all traffic flows.

4.2 DATA FLOW DIAGRAMS

4.2.1 Context Level (Level 0)

Fig 4.1 shows the system as a single process exchanging data with six external entities and one data store. Users and third-party clients send chat, session, key, memory and connector requests; the platform verifies identities with Clerk, performs completions against Ollama or cloud providers, issues search queries to Tavily, and reads and writes all five tables of `orb.db`.

4.2.2 Level 1 Decomposition

At Level 1 the platform decomposes into nine processes: authentication (1.0), session management (2.0), the agent loop (3.0), tool execution (4.0), connector key resolution (5.0), memory extraction and risk scoring (6.0), API-key and audit management (7.0), connector management (8.0) and memory management (9.0). The agent loop is the hub: it loads sessions and memories, resolves provider keys, dispatches allowed tool calls, streams to the client, fires post-chat analysis asynchronously and, for programmatic calls, logs to the audit store.

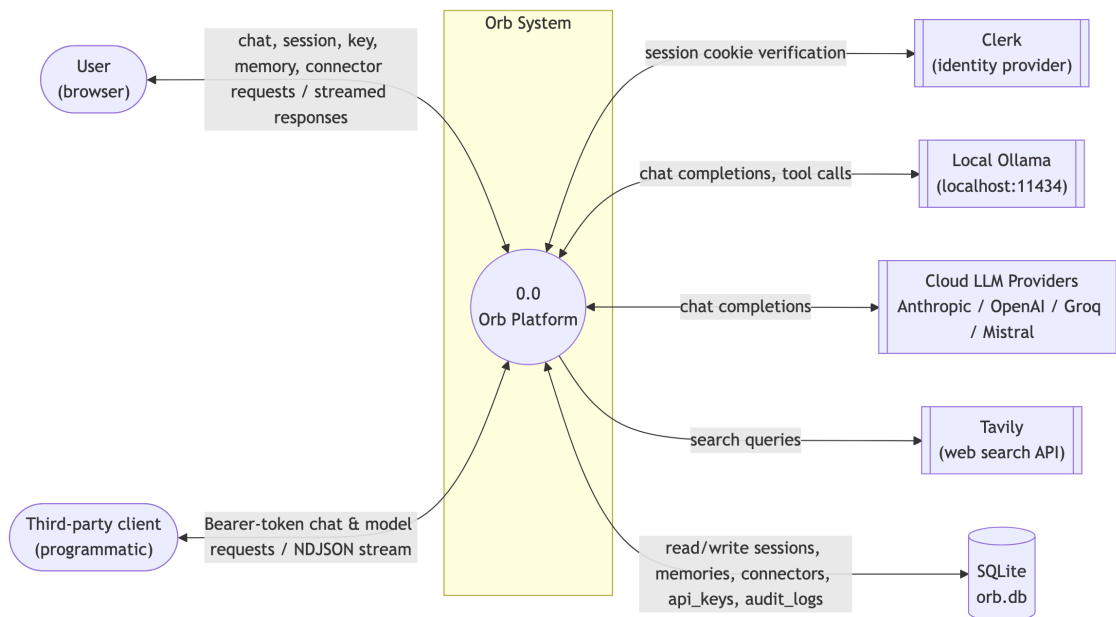


Fig 4.1 Context-level data flow diagram

4.3 DATABASE DESIGN

The schema (Fig 4.2) has five tables. Clerk is the source of truth for identity, so there is no local users table; `user_id` columns are logical references to Clerk IDs. The only enforced foreign key is `audit_logs.api_key_id → api_keys(id)`.

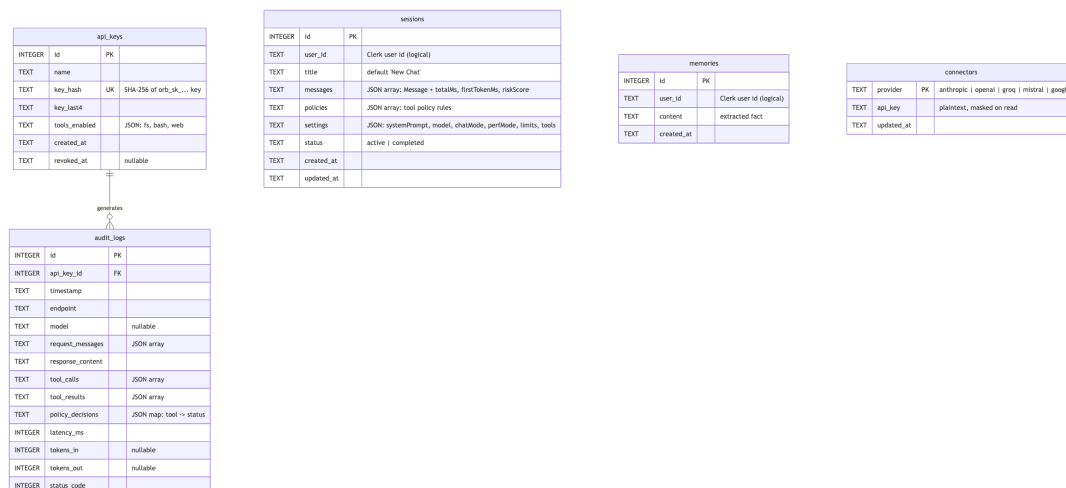


Fig 4.2 Entity-relationship diagram

Table 4.1 Database tables and their purpose

Table	Purpose
sessions	One row per conversation: title, JSON message array (with per-message latency, first-token time and risk score), JSON policy rules, JSON settings (system prompt, model, chat mode, performance mode, limits, tools), status (active/completed).
memories	Durable facts extracted per user; injected into future system prompts.
connectors	One row per provider (anthropic, openai, groq, mistral, google); stores the provider API key, upserted on save, masked on read.
api_keys	Programmatic credentials: name, SHA-256 hash, last four characters, JSON tool flags (fs, bash, web), revocation timestamp.
audit_logs	One row per /api/v1/chat call: endpoint, model, request messages, response content, tool calls and results, per-tool policy decisions, latency, token counts, status code.

4.4 PROCESS DESIGN: THE AGENT LOOP

An interactive chat request follows the sequence: the frontend posts to /api/chat; requireAuth verifies the Clerk cookie; the route loads or creates the active session and reads the user’s memory facts; a system prompt is assembled from the facts and the tool-use directive; the LLM factory instantiates the correct provider client (resolving connector keys for cloud models); and the agent loop streams model output in a reason–act–observe cycle. Whenever the model emits a tool call, the policy resolver classifies it according to the session’s chat mode — *Auto* allows all tools, *Manual* holds every call, and *Policy* applies per-tool rules: *Allowed* calls are executed by the ToolExecutor and their results fed back into the loop; *Blocked* calls are injected back as refusal messages so the model can continue; and *Requires Approval* pauses the loop and emits a requires_approval event, after which the frontend can invoke /api/execute_tool to resume. After the final reply, the route persists messages with timing data and fires postChatAnalysis asynchronously, which extracts new facts into memories and patches the message’s risk score.

The programmatic path (`/api/v1/chat`) differs in three ways: authentication is by hashed Bearer key; the allowed tool set is derived from the key's `tools_enabled` flags rather than session policies; and every call is written to `audit_logs`.

4.5 CONNECTOR KEY RESOLUTION

Cloud provider credentials are resolved by a two-level lookup (Fig 4.3): the factory first queries the `connectors` table; if no row exists it falls back to the corresponding `<PROVIDER>_API_KEY` environment variable. Database values therefore override environment configuration, allowing keys to be managed from the console at runtime without restarting the server.

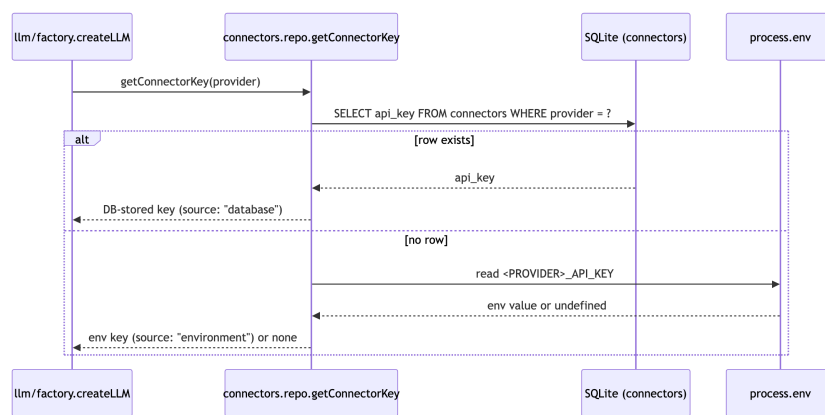


Fig 4.3 Connector key resolution sequence

4.6 API DESIGN

The gateway exposes two authentication surfaces: cookie-authenticated interactive routes and key-authenticated programmatic routes. Table 4.2 summarises the main endpoints.

4.7 USER INTERFACE DESIGN

The console is a single-page application with screen-level navigation: a cinematic *Landing* screen, the *App* (chat) screen with model selector, chat modes (*Ask / Search / Research*), policy controls and live telemetry, a *Sessions* screen listing past conversations with average latency and risk, a *Memory* screen for viewing and deleting extracted facts, an *API* screen for key management, audit logs and analytics, and a *Connectors* screen listing all five providers with config-

Table 4.2 Principal API endpoints

Method	Path	Auth	Purpose
POST	/api/chat	cookie	Stream agent run (NDJSON)
POST	/api/execute_tool	cookie	Resume after approval
POST	/api/v1/chat	API key	Programmatic agent run, audited
GET	/api/models	none	List local Ollama models
GET	/api/system-info	none	CPU/RAM detection
GET/PATCH	/api/sessions/active	cookie	Active session, policy/settings patch
GET	/api/sessions[:id]	cookie	List / inspect sessions
POST	/api/sessions/:id/resume	cookie	Reactivate a session
GET/DELETE	/api/memories[:id]	cookie	List / delete facts
POST/GET	/api/keys	cookie	Create / list API keys
DEL / PATCH	/api/keys/:id[...]	cookie	Revoke / re-scope key
GET	/api/audit-logs	cookie	Recent audit rows
GET	/api/analytics/summary	cookie	Usage analytics
GET/POST	/api/connectors	cookie	List / save provider keys
DELETE	/api/connectors/:provider	cookie	Remove stored key

uration state and masked keys. Sign-in and sign-up are Clerk-hosted components. The UI uses vanilla CSS variables for light/dark theming.

CHAPTER 5: IMPLEMENTATION

5.1 IMPLEMENTATION ENVIRONMENT

The system is organised as two Node.js applications in one repository: `backend/` (Express + TypeScript gateway, run with `tsx`, listening on port 3001) and `frontend/` (Next.js console on port 3000). SQLite storage is created on first start by `backend/src/db/init.ts` using idempotent `CREATE TABLE IF NOT EXISTS` statements. Development followed an iterative, feature-branch workflow with Git.

5.2 MODULE DESCRIPTION

5.2.1 Authentication Module

Two middlewares guard the gateway. `requireAuth` validates the Clerk session cookie on all interactive `/api/*` routes and attaches the Clerk `userId` to the request. `apiKeyAuth` handles `/api/v1/*`: it hashes the presented `orb_sk_...` token with SHA-256, looks up `api_keys.key_hash`, rejects revoked keys and attaches the key row (including its `tools_enabled` flags) to the request. Plaintext keys are shown exactly once at creation; only the hash and last four characters are stored.

5.2.2 LLM Abstraction and Factory

All providers implement a common LLM interface (`backend/src/llm/LLM.ts`) exposing a streaming chat method. `Ollama.ts` adapts the local runtime; `Anthropic.ts` wraps the official SDK; and `OpenAICompatible.ts` is a single client parameterised by base URL, reused for OpenAI, Groq and Mistral. `factory.ts` dispatches by model-name prefix (`claude-` → Anthropic, `gpt-` → OpenAI, `groq/` → Groq, `mistral/` → Mistral, anything else → local Ollama), applies performance-mode parameters (context size, prediction limits, history trimming, keep-alive) and resolves cloud credentials via the connectors repository.

5.2.3 Agent Loop and Policy Engine

`Agent.ts` implements the reason–act–observe loop: stream model output; forward content chunks to the client; on a tool call, ask the policy resolver for a status; loop with tool results until the model replies without tool calls. Interactive runs resolve policy from the session’s chat mode (`chatMode.ts`): *Auto* allows every tool, *Manual* pauses on every call with a `requires_`-approval event until the user approves via `/api/execute_tool`, and *Policy* applies per-tool rules (Allowed / Blocked / Requires Approval) configured in the console. Blocked calls are injected back into the conversation as refusal messages so the model can adapt, and are surfaced in telemetry as blocked actions. Programmatic runs derive the allowed set from the API key’s tool flags instead.

5.2.4 Tool Executor

`ToolExecutor.ts` dispatches approved calls to the registered tools: `FsTool` (read file), `WriteFileTool`, `ListDirectoryTool`, `BashTool` (shell execution) and `WebSearchTool` (Tavily). Each tool validates its arguments and returns a structured result that is appended to the conversation and streamed to the client as `tool_call_intent` and `tool_result` events.

5.2.5 Session Management Module

`sessions.route.ts` maintains one active session per user with endpoints to read and patch its policies and settings, complete it (“New Chat”), list all sessions with computed average latency and risk, inspect any session and resume a completed one. Messages are persisted with per-message `totalMs`, `firstTokenMs` and `riskScore`.

5.2.6 Memory and Risk Analysis Module

After each interactive reply, `postChatAnalysis.ts` runs asynchronously: it sends the exchange to a local Ollama model (with thinking disabled and unbounded prediction for reliable JSON extraction from thinking models such as Qwen 3) and receives new user facts and a hallucination-risk value. Facts are inserted into memories; the risk score is patched onto the

persisted message. Because the call is fire-and-forget, analysis failures never affect the chat path.

5.2.7 Connectors Module

`connectors.route.ts` and `connectors.repo.ts` manage provider keys for the five known providers. Keys are upserted per provider, masked on every read, and deletable (falling back to environment variables). Resolution order — database first, then environment — lets the console override server configuration at runtime.

5.2.8 API Keys, Audit and Analytics Module

`keys.route.ts` creates keys with per-key tool scopes, lists them masked, revokes and re-scopes them. Every `/api/v1/chat` call is written by the audit repository with the complete request and response payloads, tool activity, per-tool policy decisions, latency, token counts and status code. The analytics endpoint aggregates this log into volume, blocked/error counts, latency statistics, hourly buckets, top tools and anomalies over a selectable window.

5.2.9 Frontend Console

The console (`frontend/src/app/page.tsx`) implements screen-level navigation between Landing, Transition, App (chat), Sessions, Memory, API and Connectors screens. The chat screen consumes the NDJSON stream incrementally, renders markdown with syntax highlighting, surfaces tool-call intents and approval prompts inline, and displays live telemetry. Framer Motion and GSAP drive the animated transitions; Clerk components provide sign-in and sign-up.

CHAPTER 6: TESTING

6.1 TESTING STRATEGY

Testing combined three approaches. First, manual functional testing against the running system: each functional requirement of Chapter 3 was exercised through the console. Second, API-level verification using `curl` for the programmatic routes, with Bearer keys of varying scopes. Third, an automated model-behaviour evaluation harness (`backend/scripts/eval-tool-calls.ts`) that runs five tool-calling scenarios against a live Ollama model — imperative shell requests, question-phrased shell, web-search and file-read requests, and a pure-knowledge question expected to trigger *no* tool call — reporting pass/fail per case. Emphasis throughout was placed on the policy engine (the security-critical path), authentication boundaries and the resilience of the asynchronous analysis stage. Formal unit and integration test suites are an acknowledged gap, noted as future work.

6.2 TEST CASES

Table 6.1 lists representative test cases covering authentication, policy enforcement, connector resolution, memory extraction and failure handling, together with the observed outcomes.

6.3 TESTING OUTCOME

All representative test cases passed. The policy engine correctly distinguished Auto, Policy and Manual tools; audit rows were complete for every programmatic call; and failures injected into the analysis stage and the Ollama runtime degraded gracefully without corrupting sessions.

Table 6.1 Representative test cases

ID	Scenario	Expected result	Actual result	Status
TC-01	Unauthenticated call to /api/sessions	401 rejected	As expected	Pass
TC-02	Chat with local Qwen 3 model	Streamed NDJSON reply, message persisted with latency	As expected	Pass
TC-03	Model requests bash under Manual policy	Loop pauses with requires_approval; resumes after approval	As expected	Pass
TC-04	Model requests blocked tool	Call blocked, decision recorded, loop continues	As expected	Pass
TC-05	/api/v1/chat with key lacking bash scope	Bash tool unavailable to model; audit row written	As expected	Pass
TC-06	Revoked API key used	401 rejected	As expected	Pass
TC-07	Save Anthropic key in Connectors, then chat with Claude	DB key used (source “database”), chat succeeds	As expected	Pass
TC-08	Delete stored key with env fallback present	Source reverts to “environment”	As expected	Pass
TC-09	State a personal fact, start new session	Fact extracted to memory and injected into next system prompt	As expected	Pass
TC-10	Ollama offline during chat	Graceful error surfaced; session intact	As expected	Pass
TC-11	Post-chat analysis fails	Chat reply unaffected; no risk score patched	As expected	Pass
TC-12	Resume a completed session	Session reactivated with full history and settings	As expected	Pass

CHAPTER 7: RESULTS AND DISCUSSION

7.1 RESULTS

The completed system delivers a working governance console in which a user signs in through Clerk, converses with local or cloud models, watches tool calls being allowed, blocked or held for approval in real time, and reviews sessions, memories, audit logs and analytics from dedicated screens. Figures 7.1–7.3 show the principal screens.

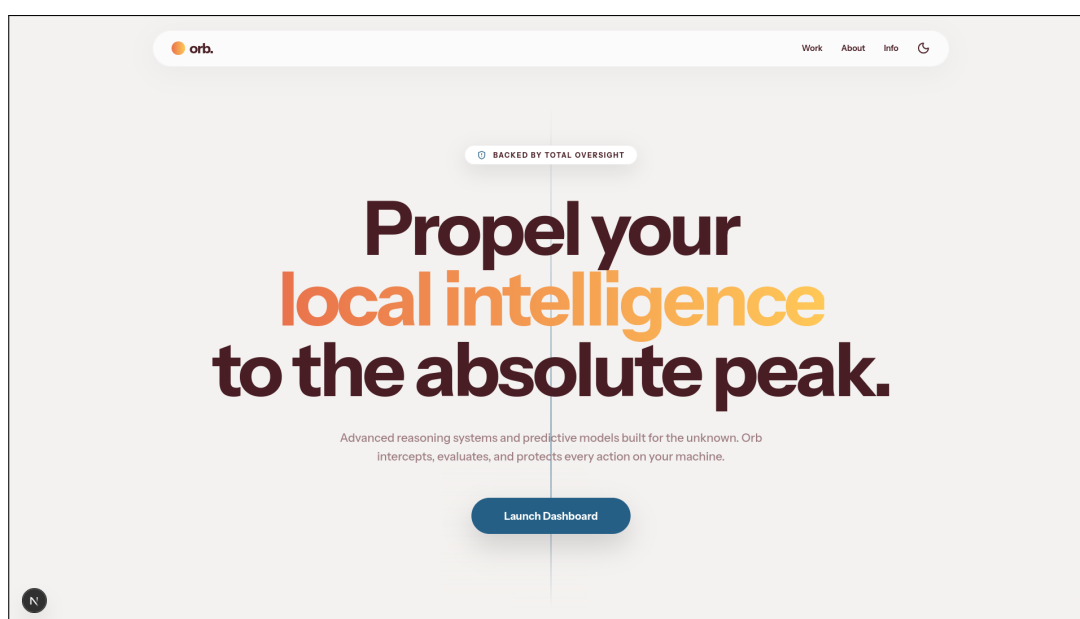


Fig 7.1 Landing screen

7.2 DISCUSSION

The central design decision — forcing all model and tool traffic through one governed gateway — proved effective: policy, audit and telemetry required no cooperation from the models themselves and work identically for local and cloud backends. Streaming NDJSON kept the interface responsive even for slow local models, and the asynchronous analysis stage added memory and risk scoring at zero perceived latency cost. The main trade-offs observed were the JSON-column denormalisation in SQLite (simple and fast for a single user, but unsuited to multi-tenant ana-

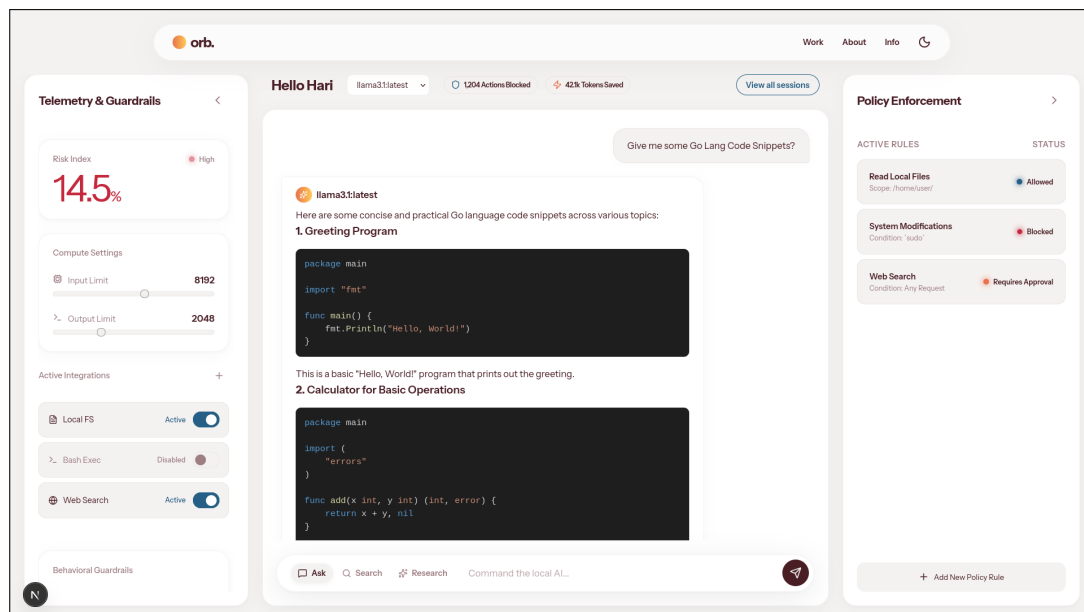


Fig 7.2 Chat console with telemetry and policy controls

lytics) and the reliance on a local model for risk scoring, whose judgement quality varies with the model used.

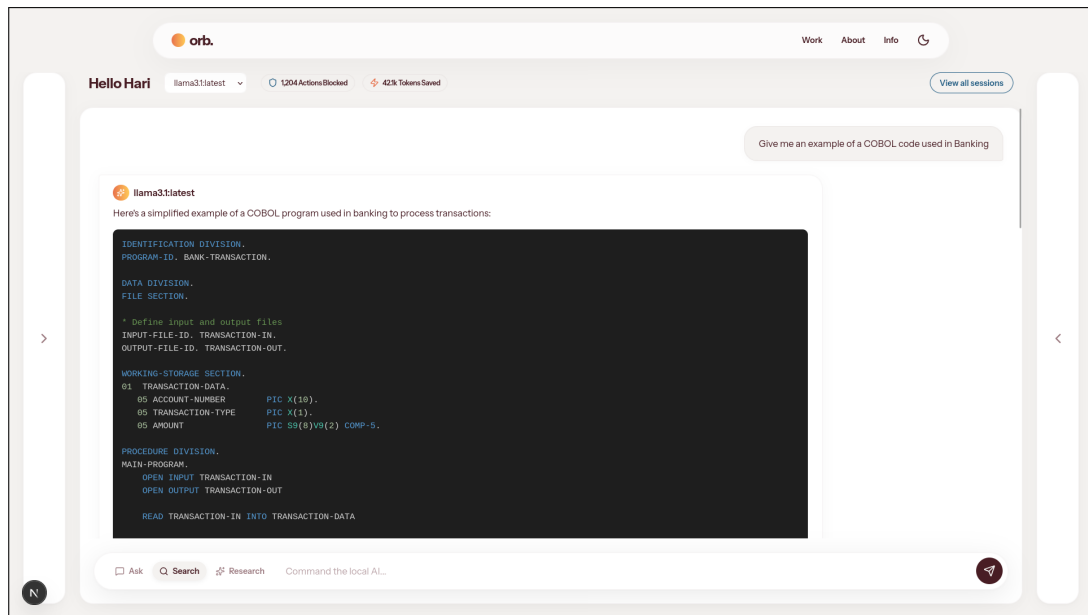


Fig 7.3 Compact chat view

CHAPTER 8: CONCLUSION AND FUTURE ENHANCEMENTS

8.1 CONCLUSION

The aim of this project was to design and implement **Orb**, a governance-first command console that places policy enforcement, auditability and telemetry between users and language models, whether those models run locally through Ollama or in the cloud through Anthropic, OpenAI, Groq or Mistral. The objectives set out in Chapter 1 were met through eight cooperating modules: Clerk- and key-based authentication, a provider-agnostic LLM abstraction with a connectors module for credential management, an agentic chat loop with a three-level tool policy engine (Auto / Policy / Manual), a governed tool executor (file system, shell, web search), persistent sessions with per-message telemetry, asynchronous memory extraction with hallucination-risk scoring, and a fully audited programmatic API with analytics.

The delivered system demonstrates that meaningful governance of agentic LLMs can be achieved at the gateway layer: every tool call is policy-checked, every programmatic call is audited, every response carries latency and risk telemetry, and user context persists across sessions — all while local-first privacy is preserved. Its principal limitations are single-machine deployment, heuristic risk scoring dependent on a local judge model, plaintext storage of provider keys in the local database, and the absence of chat support for the Google connector.

8.2 FUTURE ENHANCEMENTS

- Encrypt provider keys at rest and add OS-keychain integration.
- Complete Gemini chat support and add streaming tool use for all cloud providers.
- Multi-user, multi-tenant deployment with role-based access to policies and audit logs.

- Richer policy language: per-path file rules, command allowlists and time-boxed approvals.
- Ensemble or retrieval-grounded hallucination scoring to replace the single-judge heuristic.
- Export of audit logs to standard formats (CSV, OpenTelemetry) for external compliance tooling.
- Semantic (vector) memory retrieval in place of injecting all facts into every prompt.

REFERENCES

- [1] Sommerville, Ian. *Software Engineering*. 10th ed. Harlow: Pearson Education, 2016.
- [2] Pressman, Roger S., and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. 9th ed. New York: McGraw-Hill Education, 2020.
- [3] Banks, Alex, and Eve Porcello. *Learning React: Modern Patterns for Developing React Apps*. 2nd ed. Sebastopol: O'Reilly Media, 2020.
- [4] Vercel Inc. "Next.js Documentation." Next.js. 2026. 15 July 2026 <<https://nextjs.org/docs>>.
- [5] OpenJS Foundation. "Express – Node.js web application framework." Express. 2026. 15 July 2026 <<https://expressjs.com/>>.
- [6] Ollama. "Ollama API Documentation." Ollama. 2026. 15 July 2026 <<https://github.com/ollama/ollama/blob/main/docs/api.md>>.
- [7] Anthropic. "Anthropic API Documentation." Anthropic. 2026. 15 July 2026 <<https://docs.anthropic.com/>>.
- [8] Clerk Inc. "Clerk Documentation." Clerk. 2026. 15 July 2026 <<https://clerk.com/docs>>.
- [9] Hipp, D. Richard. "SQLite Documentation." SQLite. 2026. 15 July 2026 <<https://www.sqlite.org/docs.html>>.
- [10] Wiggins, Joshua. "better-sqlite3 API Documentation." better-sqlite3. 2026. 15 July 2026 <<https://github.com/WiseLibs/better-sqlite3>>.
- [11] Tavily. "Tavily Search API Documentation." Tavily. 2026. 15 July 2026 <<https://docs.tavily.com/>>.
- [12] Ji, Ziwei, et al. "Survey of Hallucination in Natural Language Generation." *ACM Computing Surveys* 55 12 Mar. 2023: 1-38.

```
\def \etb@resrvda{{\if@fancyplain \let \headrulewidth \plainheadrulewidth \fi  
\color {darkred}\color {darkred}\hrule \@height \headrulewidth \@width \headwidth  
\skip -\headrulewidth }}\def \etb@resrvda{{\if@fancyplain \let \footrulewidth  
\plainfootrulewidth \fi \color {darkred}\color {darkred}\hrule \@width \headwidth  
\@height \footrulewidth }}
```

APPENDIX A

USER MANUAL

A.1 INSTALLATION

1. Install Node.js 20+ and Ollama; pull at least one local model (e.g., `ollama pull qwen3`).
2. Clone the repository and install dependencies in both `backend/` and `frontend/` with `npm install`.
3. Create `backend/.env` with Clerk keys and (optionally) provider keys such as `ANTHROPIC_API_KEY` and `TAVILY_API_KEY`.
4. Start the backend (`npm run dev`, port 3001) and the frontend (`npm run dev`, port 3000), then open `http://localhost:3000`.

A.2 USING THE CONSOLE

Sign in, choose a model from the selector (local models appear automatically; cloud models appear once their connector key is configured), pick a chat mode (Ask / Search / Research) and converse. When a tool call requires approval, an inline prompt appears — the run pauses until you approve or deny. Use the Sessions screen to revisit or resume conversations, the Memory screen to curate extracted facts, the API screen to issue `orb_sk_...` keys and inspect audit logs and analytics, and the Connectors screen to store provider keys (stored keys override environment variables).

A.3 PROGRAMMATIC ACCESS

Create a key on the API screen, then call:

```
curl -N http://localhost:3001/api/v1/chat \  
-H "Authorization: Bearer orb_sk_..." \  

```

```
-H "Content-Type: application/json" \  
-d '{"model":"qwen3","messages":[{"role":"user",  
    "content":"List files in the current directory"}]}'
```

The response is an NDJSON stream; every call is recorded in the audit log with its policy decisions.

APPENDIX B

PLANNING

B.1 GANTT CHART

Table B.1 Project schedule

Phase	Period
Requirements and study of existing systems	Weeks 1–2
Architecture and database design	Weeks 3–4
Gateway, auth and chat streaming	Weeks 5–7
Policy engine and tool executor	Weeks 8–9
Sessions, memory and risk analysis	Weeks 10–11
API keys, audit, analytics, connectors	Weeks 12–13
Testing, documentation and report	Weeks 14–15

APPENDIX C

DESIGN CONCEPTS ANALYSIS

C.1 HEALTH AND SAFETY

Orb's core purpose is operational safety: it interposes human approval before a model executes shell commands or writes files, preventing unintended damage to the host system.

C.2 PRIVACY AND DATA PROTECTION

The system is local-first: sessions, memories, audit logs and keys never leave the user's machine, and local inference involves no external data transfer. Cloud providers receive only the messages the user explicitly routes to them.

C.3 SECURITY

Credentials are never stored in recoverable form for programmatic keys (SHA-256 hash only); provider keys are masked on read; dangerous tools default to manual approval; and the audit log provides after-the-fact accountability for all programmatic access.

C.4 SUSTAINABILITY AND COST

Local inference on existing hardware avoids per-token cloud costs and gives the user explicit control (token limits, performance modes) over compute expenditure.